Важно!
Чтобы продолжать учиться и учить

Подробнее

до 11 янв., 01:30

Открыть урок

Когда: 24.10.26 в 16:00

Урок 5. Цикл `for` — не будем зацикливаться на `while`. Методический материал.

- Общая информация об уроке
- Структура урока
- Цикл for по элементам
- ✅ Задача «По буковке»
- Цикл for по диапазону: range()
- ✅ Задача «Решать задачи»
- ✅ Задача «А теперь нумерация!»
- ✅ Задача «Трамплин»
- Аргументы print(): sep и end
- ✅ Задача «Классики»
- ✅ Задача «Горка букв»
- Кодирование символов: Unicode, ord(), chr()
- ✅ Задача «Расширенный Цезарь»
- Завершение урока
- Таблица задач урока

Тип урока: Новая тема
Предварительные знания учеников: Переменные, типы данных, арифметические операции, преобразование типов, ввод и вывод данных, f-строки, условный оператор `if-elif-else`, цикл `while`, сокращённые операторы присваивания (уроки 1–4)

На этом уроке ученики научатся:

- Использовать цикл `for` для перебора элементов последовательности (строк, перечислений).
- Создавать числовые последовательности с помощью `range()` с одним, двумя и тремя аргументами.
- Использовать отрицательный шаг в `range()` для обратного счёта.
- Настраивать вывод `print()` с помощью именованных аргументов `sep` и `end`.
- Применять оператор распаковки * совместно с `sep`.
- Понимать устройство кодировки Unicode.
- Использовать функции `ord()` и `chr()` для перевода между символами и их кодами.

Структура урока

№	Блок	Ориентировочное время	Задачи
1	Цикл `for` по элементам	8–10 мин	—
2	**Задача «По буковке»**	5–8 мин	Перебор букв строки, f-строка в теле цикла
3	Цикл `for` по диапазону: `range()`	9–12 мин	—
4	**Задача «Решать задачи»**	5–8 мин	`range(n)`, f-строка с переменной цикла
5	**Задача «А теперь нумерация!»**	5–8 мин	`range(start, stop)`, f-строка с двумя переменными
6	**Задача «Трамплин»**	8–10 мин	`range` с шагом и отрицательным шагом, два последовательных цикла
7	Аргументы `print()`: `sep` и `end`	5–8 мин	—
8	**Задача «Классики»**	8–10 мин	`range` с шагом, условие

№	Блок	Ориентировочное время	Задачи
10	**Задача «Горка букв»**	8–10 мин	Перебор строки с индексом, *, sep, end
11	Кодирование символов: Unicode, ord(), chr()	8–10 мин	—
12	**Задача «Расширенный Цезарь»**	8–10 мин	ord, chr, остаток от деления, ветвление на два алфавита
13	Подведение итогов	2–3 мин	—

⏱ **Темп урока:** Блок с range() — центральный, не торопитесь. Ученики, знакомые с while-счётчиком, поначалу могут не понимать, зачем нужен for. Отдельно обсудите: for не лучше и не хуже while — просто там, где количество шагов известно заранее, for компактнее и понятнее. Блок с Unicode — чуть медленнее: нужно время, чтобы «переварить» идею кодировки. Задачи «Трамплин» и «Классики» требуют внимания к граничным условиям — планируйте на них дополнительное время.

Цикл for по элементам

Время: 8–10 минут. Обязательный фундамент — объясните идею до того, как перейти к range().

Ключевые идеи для объяснения

- Напомните о цикле while: он повторяет действия, **пока выполняется условие**. Цикл for решает другую задачу: **пройти по каждому элементу последовательности**. Элементы закончились — цикл завершился.
- Общий синтаксис:

```
for переменная in последовательность:
    # тело цикла
```

  Синтаксически всё то же самое, что в if и while: двоеточие и отступ.
- Первый наглядный пример — перебор строки побуквенно:

```
for letter in "Python":
    print(letter)
```

закончились — цикл завершился.

- Можно перечислить значения вручную через запятую:

```
for num in 1, 2, 3, 4, 5:
    print(num)
```

- Важная особенность: **интерпретатор вычислит все значения в заголовке цикла ДО начала итераций**. Покажите пример с `input()` — три вызова `input()` в заголовке сработают ещё до входа в тело.
- Термин **итерация** уже знаком с прошлого урока — используйте его уверенно.

Рекомендация: Напишите пример живьём. Покажите, как переменная `letter` «идёт» по строке: можно нарисовать строку на доске и стрелкой обозначить текущее положение на каждой итерации. Подчеркните: `for` сам следит за переходом к следующему элементу, нам не нужно вручную менять счётчик — это главное удобство по сравнению с `while`.

Обратите внимание, что цикл `for` перебирает именно **элементы** последовательности, а не их **индексы**. Тем, кто знаком с другими языками программирования, это может показаться необычным.

✅ Задача «По буковке» 15 баллов

Классная **Что проверяется:** перебор строки символ за символом; f-строка в теле цикла; точка в конце каждой строки вывода.

```
word = input()
for letter in word:
    print(f"Буква {letter}.")
```

Как вести на уроке:

- Разберите задачу словами: «Считываем слово → для каждой буквы выводим строку по шаблону».
- Ключевой вопрос перед решением: «Что является последовательностью в нашем `for`?» — строка `word`.
- Типичная ошибка: написать `for word in letter` — перепутать местами переменную цикла и последовательность. Обговорите явно: переменная цикла — это одна буква (называем `letter`), последовательность — всё слово

- Проверьте, что точка стоит внутри f-строки, а не выводится отдельным `print`.

Цикл `for` по диапазону: `range()`

Время: 9–12 минут. Это самый насыщенный теоретический блок урока.

Ключевые идеи для объяснения

- Что делать, если нужно повторить действие **n раз**, а не перебирать символы строки? Можно вручную перечислить числа — но это неудобно. Для этого есть `range()`.
- `range(5)` создаёт последовательность **0, 1, 2, 3, 4** — не до 5, а до 4. Счёт с нуля — программистское соглашение.

```
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4
```

- Если переменная цикла не нужна в теле — по PEP 8 её можно назвать `_`:

```
for _ in range(100):
    print("Я выучу Python")
```

- Чтобы считать с единицы — два способа:
  1. Прибавить 1 в теле: `print(i + 1)`
  2. Задать начало диапазона: `range(1, n + 1)`

  Покажите оба варианта.
- `range()` с тремя аргументами `range(start, stop, step)`:
  - `range(start, stop)` — от `start` включительно до `stop` не включительно
  - `range(start, stop, step)` — то же, но с шагом `step`
  - `range(start, stop, -1)` — обратный счёт (`start > stop`)

```
for countdown in range(10, 0, -1):
    print(f"🚀 {countdown}...")
print("Ту зе мун!")
```

Итоговая таблица аргументов — напишите на доске:

Вызов	Описание
`range(stop)`	от 0 до `stop` (не включая `stop`)

Рекомендация: Проведите живую демонстрацию: введите `range(5)` в терминале — ученики увидят `range(0, 5)`, а не список чисел. Это хороший повод объяснить, что `range()` — это не список, он не создаёт все числа сразу, а выдаёт их по одному. Поэтому `range(1_000_000_000)` не «съест» память. Если есть время, покажите дополнительные возможности: `len(my_range)`, `my_range.start`, `2 in my_range`. **Важно:** при отрицательном шаге начало (`start`) должно быть больше конца (`stop`), иначе цикл не выполнится ни разу. Убедитесь, что ученики это понимают.

✅ Задача «Решать задачи» 10 баллов

Классная **Что проверяется:** `range(n)` для выполнения `n` повторений; f-строка с переменной `n`; понимание, что тело цикла не меняется от итерации к итерации.

```
n = int(input())
for i in range(n):
    print(f"Я буду решать задачи {n} раз!")
```

Как вести на уроке:

- Обратите внимание: `i` в теле цикла **не используется** — тело одинаково на каждой итерации. Спросите у класса: «Почему тогда нельзя назвать переменную _?» — формально можно, `i` здесь для наглядности.
- Типичная ошибка: написать `{i}` вместо `{n}` в f-строке, потому что `i` — видимая переменная цикла. Обговорите разницу: `n` — сколько раз, `i` — номер текущей итерации (начиная с 0).
- Граничный случай: `n = 0` — цикл не выполнится, вывода нет.

✅ Задача «А теперь нумерация!» 15 баллов

Классная **Что проверяется:** `range(start, stop + 1)` для включительного конца; f-строка с двумя переменными; ввод нескольких значений до цикла.

```
word = input()
start = int(input())
stop = int(input())
for i in range(start, stop + 1):
```

Как вести на уроке:

- Ключевой вопрос: почему `stop + 1`? Потому что `range()` не включает верхнюю границу. Запишите на доске: «хочу до 5 включительно → пишу `range(..., 6)`».
- Типичная ошибка: забыть `+ 1` и получить вывод без последнего элемента. Специально покажите, что будет без `+ 1`, пусть ученики сами найдут ошибку.
- Порядок ввода: сначала слово, потом два числа. Ученики иногда переставляют — проговорите явно.

✅ Задача «Трамплин» 15 баллов

Классная **Что проверяется:** `range` с шагом 2; отрицательный шаг; два последовательных цикла; `end=' '` для вывода в одну строку.

```
start = int(input())
stop = int(input())
for i in range(start, stop, 2):
    print(i, end=' ')
for i in range(stop, start - 1, -1):
    print(i, end=' ')
```

Как вести на уроке:

- Разберите задачу в два этапа: сначала чётные числа от `start` до `stop` с шагом 2 (не включая `stop`), потом все числа от `stop` обратно до `start` включительно.
- Граничное условие второго цикла: `range(stop, start - 1, -1)` — чтобы включить `start`, нижняя граница `start - 1`.
- Аргумент `end=' '` здесь используется впервые — объясните его кратко, детально разберёте в следующем теоретическом блоке.
- Типичные ошибки:
  - Не дойти до `start` во втором цикле (написать `start` вместо `start - 1` в качестве нижней границы).
  - Перепутать порядок аргументов при отрицательном шаге.
- Разыграйте на числах 2 и 8: первый цикл даёт 2, 4, 6 (не 8!), второй — 8, 7, 6, 5, 4, 3, 2.

Время: 5–8 минут.

Ключевые идеи для объяснения

- По умолчанию `print()` ставит пробел между аргументами и переводит строку в конце. Это поведение можно изменить.
- Именованный аргумент `sep` (*separator*) — разделитель между несколькими аргументами в одном `print()`:

```
print("Cats", "Dogs", "Pythons", sep=" vs ")
# Cats vs Dogs vs Pythons
```

- Именованный аргумент `end` — что добавляется в конце вывода вместо `\n`:

```
print("Привет", end=", ")
print("пользователь", end="!\n")
# Привет, пользователь!
```

- По PEP 8: вокруг = в именованных аргументах **пробелы не нужны**: `end='\n'`, не `end = '\n'`.
- Оператор распаковки * заставляет `print()` воспринимать строку или `range()` как набор отдельных элементов:

```
print(*'Привет', sep='-')   # П-р-и-в-е-т
print(*range(1, 6), sep=', ')  # 1, 2, 3, 4, 5
```

Рекомендация: Покажите эффект * на живом примере: сначала `print('Python')`, потом `print(*'Python')` — видно разницу. Аналогично с `range`: `print(range(5))` против `print(*range(5))`. Подчеркните: * — это про распаковку, не умножение. Здесь он используется в особом контексте; подробности будут позже.

✅ Задача «Классики» 15 баллов

Классная **Что проверяется:** `range` с тремя аргументами; определение последнего элемента через условие; `end=', '` для всех элементов, кроме последнего.

```
start = int(input())
stop = int(input())
step = int(input())
```

```
    else:
        print(i)
```

Как вести на уроке:

- Ключевая сложность: как понять, что текущий элемент — **последний**? Проверить, войдёт ли следующий шаг в диапазон: `if i + step <= stop`.
- Разберите пример: `start=1, stop=10, step=3` → последовательность 1, 4, 7, 10. После 10 следующий элемент был бы 13 > 10 — значит, 10 последний.
- Типичная ошибка: использовать <= вместо < или забыть про `stop + 1` в `range()`.

✅ Задача «Горка букв» 15 баллов

Классная **Что проверяется:** перебор строки с ручным счётчиком; умножение строки на число; распаковка *; sep и end; разный символ в конце последней строки.

```
word = input()
i = 0
for letter in word:
    if i != len(word) - 1:
        print(*(letter * (i + 1)), sep='-', end=';\n')
    else:
        print(*(letter * (i + 1)), sep='-', end='.\n')
    i += 1
```

Как вести на уроке:

- Разберите по частям: `letter * (i + 1)` — строка из (i+1) одинаковых букв; `*(letter * (i + 1))` — распаковка этой строки в отдельные символы; `sep='-'` — соединяем их через тире.
- Обратите внимание: `i` — ручной счётчик, его нужно инициализировать до цикла и увеличивать в конце тела. Здесь `for` перебирает элементы, но нам дополнительно нужен индекс.
- Типичная ошибка: забыть `i += 1` и получить одинаковые строки на всех итерациях.
- Условие последней буквы: `i != len(word) - 1` — проверяем, дошли ли до

- Разыграйте на слове «кот»: к → к-к → к-к-к… нет, к — одна буква, о-о — две, т-т-т — три. Запишите ожидаемый вывод на доске.

Кодирование символов: Unicode, `ord()`, `chr()`

Время: 8–10 минут.

Ключевые идеи для объяснения

- В памяти компьютера любой символ хранится как **число**. Чтобы все программы понимали друг друга, нужна единая таблица соответствий.
- Кратко история: ASCII (128 символов, только латиница и цифры) → расширенная ASCII (256 символов, войны кодировок) → **Unicode** (> 150 000 символов, все языки, эмодзи, клингонский).
- В Python 3 все строки юникодные — `'Привет, 世界! 🌍'` работает из коробки.
- `ord(символ)` — возвращает Unicode-код символа:

```
print(ord('A'))   # 65
print(ord('Б'))   # 1041
```

- `chr(код)` — возвращает символ по его коду:

```
print(chr(65))    # A
print(chr(1041))  # Б
```

- Практическое применение — обход алфавита:

```
for i in range(26):
    print(chr(ord('A') + i), end=' ')
# A B C D ... Z
```

Рекомендация: Расшифруйте заголовок параграфа учебника прямо на уроке — это хороший «крючок». Подчеркните: коды латинских и кириллических букв идут **подряд**, поэтому сдвиг `ord(буква) + n` даёт предсказуемый результат внутри одного алфавита. Именно на этом строится шифр Цезаря. Но если выйти за пределы алфавита — попадём в другие символы, поэтому нужен остаток от деления (скоро покажем в задаче).

Упомяните особенность русского алфавита: буква Ё (ё) стоит не рядом с

✅ Задача «Расширенный Цезарь» 15 баллов

Классная **Что проверяется:** `ord()`, `chr()`; формула сдвига с остатком `% 32` / `% 26`; ветвление на два алфавита.

```
lang = input()
step = int(input())
word = input()
for letter in word:
    if lang == 'РУС':
        number = (ord(letter) - ord('A') + step) % 32
        print(chr(ord('A') + number), end='')
    else:
        number = (ord(letter) - ord('A') + step) % 26
        print(chr(ord('A') + number), end='')
```

Как вести на уроке:

- Разберите формулу пошагово:
  1. `ord(letter) - ord('A')` — позиция буквы в алфавите (от 0)
  2. `+ step` — сдвигаем
  3. `% 32` (или `% 26`) — «оборачиваем» назад в алфавит, если вышли за пределы
  4. `chr(ord('A') + number)` — получаем зашифрованную букву
- Запишите на доске пример: буква 'А', шаг 33, русский алфавит: `(0 + 33) % 32 = 1` → 'Б'. Без остатка от деления получили бы `33` → неверный символ.
- Объясните, почему `% 32` для кириллицы: 32 буквы без учёта Ё.
- Типичная ошибка: не учесть разницу `ord('A')` (кириллица) vs `ord('A')` (латиница) — это разные коды!
- Проверьте ввод: `lang` вводится первым, строка символов — последней. Ученики иногда путают порядок.

Завершение урока

Время: 2–3 минуты.

Что обязательно сказать

- Сегодня появился второй вид цикла — `for`. Он не лучше и не хуже `while`: когда количество итераций известно заранее — `for` компактнее; когда нужно ждать определённого события — `while`. Хороший программист умеет выбирать.
- Напомнить про **классные и домашние задачи**: если что-то не успели — продолжить дома.
- **Домашнее задание:** дочитать урок в учебнике, выполнить домашние задачи в ЛМС.
- Напомнить, где задавать вопросы: чат задачи в ЛМС и групповой чат.

Таблица задач урока

Классная работа

Задача	Баллы	Что проверяется	На что обратить особое внимание
По буковке	15	Перебор строки символ за символом; f-строка в теле цикла	Не перепутать переменную цикла и последовательность (`letter` vs `word`). Точка в конце — внутри f-строки
Решать задачи	10	`range(n)` для n повторений; f-строка с `n`, не с `i`	Переменная цикла `i` не используется в теле. Граничный случай: `n = 0` — вывода нет
А теперь нумерация!	15	`range(start, stop + 1)`; включительный конец диапазона	Обязательный `+ 1` для включения `stop`. Порядок ввода: сначала слово, потом числа
Трамплин	15	Два последовательных цикла; `range` с шагом 2 и `-1`; `end=' '`	Нижняя граница второго цикла `start - 1` (чтобы включить `start`). Первый цикл не включает `stop`
Классики	15	`range` с тремя аргументами; определение последнего элемента через `i + step <= stop`	Не забыть `stop + 1` в `range()`. Условие последнего элемента — ключевая сложность
Горка букв	15	Перебор строки с ручным счётчиком `i`; `letter * (i +`	`i += 1` в конце тела цикла. Условие последней буквы `i`

Задача	Баллы	Что проверяется	На что обратить особое внимание
Расширенный Цезарь	15	`ord()`, `chr()`; формула сдвига с остатком `% 32` / `% 26`; ветвление на два алфавита	`ord('A') ≠ ord('A')` — разные коды. Остаток от деления обязателен для «оборачивания»

Домашние задачи

Задача	Баллы	Что проверяется	На что обратить особое внимание
Блины	20	`range(n, 0, -k)` — обратный счёт с нестандартным шагом; `end=' '`	Шаг `-k`, а не `-1`. Цикл завершается, когда `i` станет ≤ 0. Последний выводимый элемент может не совпадать с k
Буквы в числах	20	`range(n)` для n повторений; `chr(code)` внутри цикла; `end=''` для вывода без пробелов	`chr()` принимает целое число — обязательно `int(input())`. Ввод n, потом n кодов
Цена удовольствия	20	`range(n)` для накопления суммы; `round(s / n, 2)` для среднего	Деление `s / n` выполняется вне цикла, а не внутри. `round(..., 2)` — ровно два знака после запятой
Decode по-цезарски	20	`ord()`, `chr()`; формула расшифровки: вычитание шага вместо прибавления; `% 32` / `% 26`	Знак минус вместо плюса — симметрия с «Расширенным Цезарем». Остаток от деления остаётся обязательным (Python корректно обрабатывает отрицательные остатки)
Направление	20	Ветвление `if number1 <= number2` для выбора направления `range`; разные шаги в зависимости от направления	В ветке «убывания»: `range(number1, number2 - 1, -step)`. Ученики часто путают знак шага или граничные значения